Homework 4
Digital Logic Circuit Simulator

COMS 2280 Fall 2025

Due at **11:59 PM, Sunday, Dec. 07**

Contents

1 Homework Overview **2**

2 Core Concepts **2**
- 2.1 The Logic Tree Structure . . . 2
- 2.2 Gate Types and Evaluation . . . 2
- 2.3 Input Format . . . 2
  - 2.3.1 Line 1: The Circuit Definition . . . 2
  - 2.3.2 Line 2+: Input Values . . . 3

3 Assignment Tasks **3**
- 3.1 Class Design Constraints . . . 3
- 3.2 To Do . . . 3

4 Class and Method Descriptions **3**
- 4.1 `CircuitTree` . . . 3

5 Testing **4**

6 Submission **4**
- 6.1 Deliverables . . . 4
- 6.2 Important Instructions . . . 4

7 Significant Dates **4**

8 Extra Credit (5% or 15%) **5**
- 8.1 Option A: Statistics (5%) . . . 5
- 8.2 Option B: Iterative Construction (15%) . . . 5

1 Homework Overview

The objective of this exercise is to implement a **Digital Logic Circuit Simulator** using a binary tree structure. Unlike the previous assignment where you managed a linear list of pages, here you will manage a hierarchical tree of logic gates.

You will write a program that reads a file containing a circuit definition (encoded as a tree in prefix notation) and a series of input values. The program must construct the circuit in memory and evaluate the output (0 or 1) for the provided inputs.

Important: Unlike previous assignments, **no skeleton code is provided** other than a basic `pom.xml` for building. You are responsible for designing your class hierarchy, provided you adhere to the public API specifications required for grading.

2 Core Concepts

This assignment relies on the concept of an Expression Tree, where internal nodes represent operations (Logic Gates) and leaf nodes represent operands (Input Variables).

2.1 The Logic Tree Structure

The circuit is represented as a tree. The flow of data moves from the leaves (bottom) up to the root (top).

- **Leaf Nodes:** These represent input variables (e.g., `A`, `B`, `C`). They hold a boolean state (0 or 1).
- **Internal Nodes:** These represent logic gates. They have child nodes (inputs) and produce a result based on their specific logic operation.
  - **Binary Gates** (AND, OR, XOR) have strictly two children (Left, Right).
  - **Unary Gates** (NOT) have strictly one child (Left).

2.2 Gate Types and Evaluation

Your tree must support the following standard logic gates:

Symbol	Gate	Children	Logic
`&`	AND	2	Returns 1 if **both** children are 1. Else 0.
`|`	OR	2	Returns 1 if **at least one** child is 1. Else 0.
`^`	XOR	2	Returns 1 if children are **different**. Else 0.
`!`	NOT	1	Returns the **inverse** of the child ($0 \to 1, 1 \to 0$).

Evaluation: The value of the circuit is the value of the root node. To find this, you must perform a **post-order traversal**:

1. Evaluate the Left Child.
2. Evaluate the Right Child (if applicable).
3. Apply the Gate Logic to the results.

2.3 Input Format

The input file (`.txt`) consists of two parts: the **Circuit Definition** and the **Input Lists**.

2.3.1 Line 1: The Circuit Definition

The circuit is provided in **Prefix Notation** (Preorder Traversal).

- **Example:** `& | A B ! C`
- **Parsing Logic:**
  - `&` (AND): Internal Node. Needs 2 children.

  - Left Child → `|` (OR): Internal Node. Needs 2 children.
  - ... Left → `A` (Var): Leaf.
  - ... Right → `B` (Var): Leaf.
  - Right Child → `!` (NOT): Internal Node. Needs 1 child.
  - ... Left → `C` (Var): Leaf.
- **Resulting Equation:** (A OR B) AND (NOT C)

2.3.2 Line 2+: Input Values

Subsequent lines contain assignments for the variables.

- Example: `A=1 B=0 C=1`

3 Assignment Tasks

3.1 Class Design Constraints

You must design your own classes in the `edu.iastate.cs2280.hw4` package. However, to ensure the autograder works, your main class must be named `CircuitTree` and must contain specific public methods.

You are strongly encouraged to use **Polymorphism** for your nodes. For example, an abstract class `LogicNode` extended by `AndNode`, `OrNode`, `VarNode`, etc., is cleaner than a single node class with a giant switch statement.

3.2 To Do

- **Tree Construction:** Read the first line of the file and construct the `CircuitTree` using a recursive (or iterative) approach.
- **Evaluation:** Parse subsequent lines to update the values of the Leaf Nodes, then evaluate the tree to find the result.
- **Input Handling:** Your program must handle variable names of arbitrary length (e.g., `Var1`, `Input_A`).
- **Main Method:** Your `main` should take a **filename as a command-line argument**, print the tree structure, and then print the result for each input line.

4 Class and Method Descriptions

4.1 `CircuitTree`

This is the public interface for your assignment.

- `public CircuitTree(String circuitDescription)`
  - Constructor. Takes the single string representing the preorder traversal (e.g., `"& | A B ! C"`).
  - Parses the string and builds the entire tree structure.
  - *Note:* Use a `Scanner` to parse tokens. If the token is a gate, create a gate node and recurse. If it is a variable, create a leaf.
- `public void evaluate(String inputLine)`
  - Takes a string of variable assignments (e.g., `"A=1 B=0 C=1"`).
  - Updates the specific `VarNode` leaves in your tree with these values.
  - Recalculates the output of the root node.
  - Prints the result to the console in the format: `A=1 B=0 C=1 -> 0`
- `public void printStructure()`
  - Prints the tree structure to the console using ASCII art to show relationships.
  - You are required to implement a "directory style" tree visualization.

- *Hint:* You will need a private recursive helper method that takes the current node and a `String` prefix.

5 Testing

Given a file `circuit.txt`:

```
^ & A B | C D
A=0 B=0 C=0 D=0
A=1 B=1 C=0 D=0
A=1 B=1 C=1 D=0
```

Logic: (A AND B) XOR (C OR D)

Running `java edu.iastate.cs2280.hw4.CircuitTree circuit.txt` should output:

```
CIRCUIT STRUCTURE:
\-- ^
    |-- &
    |   |-- A
    |   \-- B
    \-- |
        |-- C
        \-- D

EVALUATION:
A=0 B=0 C=0 D=0 -> 0
A=1 B=1 C=0 D=0 -> 1
A=1 B=1 C=1 D=0 -> 0
```

6 Submission

Write your classes in the `edu.iastate.cs2280.hw4` package.

6.1 Deliverables

- A zip file containing all the Java source files. Please do not turn in any generated `.class` files.
- The zip should include the `src` folder and `pom.xml` at its root.
- The zip file should be named `FirstName_LastName_HW4.zip`.

6.2 Important Instructions

1. Write Javadoc for documenting your homework.
2. Do not change the `CircuitTree` constructor or method signatures described above.

7 Significant Dates

The homework is due on **Sunday, Dec. 07** at 11:59 PM.

- **Extension:** You may use a **2-day extension** (until Dec 09) without any penalty or need for permission.
- **Late Submission:** You may submit **1 day late** (until Dec 10) with a penalty of 10%.
- **Hard Deadline:** No submissions will be accepted after Dec 10.
- **Early Submission:** There is no early submission bonus.

8 Extra Credit (5% or 15%)

8.1 Option A: Statistics (5%)

To earn this bonus, you must implement an additional public method to analyze the tree structure.

- **Method Signature:** `public void printStatistics()`
- **Behavior:** This method should traverse the tree (recursion is allowed here) to calculate the metrics and print them to the console.
- **Expected Output Format:**

```
STATISTICS:
Total Gates: 7
Total Inputs: 8
Max Depth: 4
```

8.2 Option B: Iterative Construction (15%)

This bonus is non-cumulative (you get 15% total, not 5% + 15%).

The standard solution for building a tree from a prefix string uses recursion. To earn the 15% bonus, you must implement the `CircuitTree` constructor using an **Iterative** approach, avoiding recursion entirely during the build phase.

- **Method Signature:** You must use the **same standard constructor** defined in Section 4.1:
  - `public CircuitTree(String circuitDescription)`
- **Constraint:** Inside this constructor, you must **NOT** use recursion (neither directly nor via recursive helper methods). You must explicitly use a `Stack` to manage the tree construction state.
- If you attempt this, put a comment immediately after the `@author` indicating you did the iterative construction.